The flock.
Tech talent, when you need it

The flock.

Managed Software Teams

TECHNICAL CHALLENGE

Twitter Clone

Full-Stack Challenge

1. Overview

Este challenge evalúa tu capacidad para desarrollar una aplicación full-stack completa: un clon funcional de Twitter/X. El dominio es conocido por todos, lo que te permite enfocarte en la calidad técnica y en la toma de decisiones de arquitectura sin tiempo perdido entendiendo el negocio.

> *Esperamos que uses herramientas de AI/agentic coding (Cursor, Codex, Claude Code, GitHub Copilot, o similares) durante todo el desarrollo. Parte central de la evaluación es tu capacidad de dirigir estas herramientas de manera efectiva para producir software de calidad profesional.*

2. Stack Tecnológico

El stack es completamente libre. Podés usar cualquier combinación de lenguajes, frameworks y bases de datos que consideres adecuada. Lo único que requerimos:

- Al menos un backend con una base de datos relacional (PostgreSQL o SQLite preferidos).
- Al menos un frontend web con rendering del lado del cliente o del servidor.
- Autenticación propia**:** no usar soluciones de terceros como Firebase Auth o Supabase Auth.
- La elección de stack debe estar justificada en el README.

Ejemplos: Rails + React, Django + HTMX, .NET MVC + Razor + Alpine.js, Node.js + Vue, Go + Templ, Laravel + Livewire. No se punta el stack elegido, se punta la ejecución.

3. Importante: Observamos el Proceso

Vamos a analizar no solo el resultado final, sino todo el proceso de desarrollo. En particular:

- **El repositorio GitHub:** debe ser público (o compartido con los evaluadores). Todo el historial de desarrollo debe estar visible.
- **Los commits:** no hagas squash. Queremos ver cada commit individual, su mensaje, su tamaño y su coherencia. Los commits deben contar la historia de cómo construiste la aplicación.
- **La evolución:** esperamos ver una progresión lógica, desde el scaffolding inicial hasta features completas. No un solo commit gigante al final.
- **Patrones de uso de AI:** un buen uso de agentic coding se nota en la consistencia de patterns, buena estructura desde el arranque, y velocidad de iteración.

4. Features Obligatorias

4.1 Autenticación

- Registro de usuario (email + password como mínimo).
- Login / Logout.
- Protección de rutas: las acciones autenticadas deben requerir sesión válida.
- Perfil básico de usuario: nombre de usuario único, bio y avatar placeholder.

4.2 Tweets

- Crear tweet (texto, máximo 280 caracteres).
- Eliminar tweet propio.
- Timeline: mostrar tweets de los usuarios seguidos, ordenados cronológicamente.
- Paginación o infinite scroll en el timeline.

4.3 Interacciones Sociales

- Follow / Unfollow de usuarios.
- Like / Unlike en tweets.
- Contador de likes visible en cada tweet.
- Listado de followers y following en el perfil de usuario.

4.4 Búsqueda

- Búsqueda básica de usuarios por nombre o username.

4.5 Responsive Design

- La aplicación debe ser completamente responsive y usable en mobile.

- **Approach mobile-first:** el diseño debe pensarse primero para mobile y escalar a desktop.
- Breakpoints mínimos: mobile (< 640px), tablet (640–1024px), desktop (> 1024px).

5. Requerimientos Técnicos

5.1 Testing

Cobertura mínima de tests del 80% en el backend. Esto incluye:

- Unit tests para modelos y validaciones.
- Tests de integración para endpoints críticos de la API.
- Al menos un test end-to-end del flujo de autenticación.

En el frontend, incluir al menos tests de integración para los flujos principales (login, crear tweet, follow).

5.2 Seed Data

- El proyecto debe incluir un seed que genere datos realistas: al menos 10 usuarios con tweets, follows y likes cruzados.
- Debe ser posible levantar el proyecto y ver contenido inmediatamente después de correr el seed.

5.3 README y Runbook

> *El Runbook es obligatorio y será lo primero que lean los evaluadores. Si la aplicación no puede levantarse siguiendo el Runbook sin intervención manual adicional, se descuenta directamente de la nota de Funcionalidad.*

Runbook (Setup & Operación):

- Prerrequisitos: versiones exactas de runtime, herramientas y dependencias necesarias (ej: Node 20, Ruby 3.3, .NET 8, Docker 25, etc.).
- Pasos de instalación: comandos exactos, en orden, para clonar el repo y dejarlo listo para correr. Sin pasos implícitos ni "configurá tu entorno".
- Cómo correr el seed de datos.
- Cómo levantar la aplicación en modo desarrollo.
- Cómo correr la suite de tests completa.
- Variables de entorno requeridas: listado completo con descripción y valores de ejemplo (un archivo .env.example es obligatorio).

- Credenciales de acceso de ejemplo para probar la app una vez levantada (al menos un usuario del seed con email y password).

Decisiones técnicas:

- Por qué elegiste el stack que elegiste.
- Cómo modelaste el timeline y el grafo de follows.
- Cómo manejaste la autenticación.
- Trade-offs y limitaciones conocidas.
- Qué herramientas de AI usaste y cómo las aprovechaste.

6. Features Opcionales (Bonus)

Elegí al menos una o dos de las siguientes. Estas suman puntos y dan señal de tu capacidad de manejar complejidad adicional con herramientas de AI:

- **Real-time updates:** actualización en tiempo real del timeline (WebSockets o SSE).
- **Upload de imágenes:** permitir adjuntar una imagen al tweet.
- **Reply threads:** respuestas a tweets con visualización en hilo.
- **Notificaciones:** sistema básico de notificaciones (nuevo follow, like en tu tweet, etc.). Nota: este feature es intencionalmente ambiguo — las decisiones de diseño son parte de la evaluación.
- **Docker:** docker-compose que levante todo el stack con un solo comando.

7. Entrega

7.1 Formato

- **Repositorio GitHub:** crear un repositorio (público o con acceso para los evaluadores) y compartir el link.
- **Commits sin squash:** todo el historial debe estar visible. No se acepta un solo commit con todo el código.
- **Branch principal:** el código entregado debe estar en main o master.

7.2 Plazo

El plazo de entrega es de 72 horas calendario desde que recibís este documento. Si necesitás una extensión, comunicálo con anticipación.

7.3 Qué esperamos ver en el repo

- Primer commit: scaffolding del proyecto.
- Commits progresivos: features implementadas una a una, con mensajes descriptivos.
- Commits de testing: tests agregados junto con las features, no todos al final.
- Últimos commits: polish, documentación y cleanup.

8. Rúbrica de Evaluación

Criterio	Peso	Qué evaluamos
Funcionalidad	25%	Todas las features obligatorias funcionan correctamente. La app es usable de punta a punta.
Calidad de código	20%	Código limpio, consistente, bien organizado. Naming claro. Patterns adecuados al stack elegido.
Testing	25%	Cobertura adecuada en el backend (85%+). Tests significativos en el frontend. Tests que realmente validen lógica, no tests vacíos.
Proceso de desarrollo	15%	Historial de commits coherente. Progresión lógica. Evidencia de buen uso de herramientas de AI.
Documentación	10%	README completo con Runbook operativo. Decisiones de arquitectura explicadas. Instrucciones claras de setup.
Bonus features	5%	Features opcionales implementadas correctamente. Puntos extra por complejidad y calidad.

9. Nota Final

Este challenge está diseñado para ser completado con asistencia de herramientas de agentic coding. El scope es intencionalmente amplio para evaluar tu capacidad de dirigir estas herramientas de manera efectiva, tomar decisiones técnicas, y entregar software de calidad profesional en un tiempo acotado.

No buscamos perfección, buscamos pragmatismo inteligente: saber qué delegar a la AI, qué revisar a mano, y cómo iterar rápido sin sacrificar calidad.

Tené en cuenta que la elección de stack es parte de la evaluación — no por el stack en sí, sino por cómo lo justificás y cómo lo ejecutás.

¡Buena suerte!